

Quantum Computing: k-Coloring Problem with Range Oracle

An Insightful Approach to Constraint Satisfaction in Quantum Algorithms

Lorenzo Arcangeli
*Department of Electronics,
Information and Bioengineering
Politecnico di Milano
Milano, Italy
lorenzo.arcangeli@mail.polimi.it*

Simone Reale
*Department of Electronics,
Information and Bioengineering
Politecnico di Milano
Milano, Italy
simone.l.reale@polimi.it*

Elisabetta Di Nitto
*Department of Electronics,
Information and Bioengineering
Politecnico di Milano
Milano, Italy
elisabetta.dinitto@polimi.it*

Abstract—Quantum computing has emerged as a promising paradigm for solving computationally intensive problems, including those in graph theory and optimization. The k-coloring problem, which is NP-complete, challenges researchers to assign at most k colors to graph vertices such that no two adjacent vertices share the same color. This paper explores the use of Grover’s algorithm, a quantum search algorithm, for addressing the k-coloring problem.

A specific focus is placed on the implementation of a ”Range Oracle,” a method introduced to simplify the oracle complexity typically required in quantum graph coloring. By leveraging this approach, the paper demonstrates a reduction in circuit depth and width, contributing to the larger conversation on quantum resource optimization. This work is presented as a modest contribution to the field, with the aim of clarifying methodologies and identifying future research directions.

Index Terms—Quantum computing, k-coloring, Range Oracle, Constraint satisfaction, Grover’s algorithm, Circuit optimization

I. INTRODUCTION

Quantum computing leverages the principles of quantum mechanics, such as superposition and entanglement, to process information in ways that are fundamentally different from classical computing. It holds promise for solving problems that are computationally infeasible for classical computers, particularly in fields such as cryptography, optimization, and machine learning.

One specific area of interest is graph theory, where problems like the k-coloring problem are notoriously challenging due to their NP-complete nature. The k-coloring problem has practical applications in scheduling, register allocation, and frequency assignment. It requires assigning at most k colors to the vertices of a graph such that no two adjacent vertices share the same color.

Classical methods for solving the k-coloring problem often rely on heuristic algorithms or exhaustive search, both of which become computationally expensive as the graph size grows. Quantum computing introduces the possibility of leveraging quantum parallelism to achieve quadratic speedups using Grover’s algorithm. However, the practical implementation of

Grover’s algorithm for graph coloring faces several challenges, including the complexity of oracle design and the scalability of quantum circuits.

This paper aims to address these challenges by introducing a Range Oracle methodology, which simplifies oracle complexity and improves resource efficiency. The goal is to provide an incremental improvement to existing quantum approaches optimizing the invalid color detection phase, which is the first operation done on the nodes of the graph, checking that the color assigned to the node is actually existing among the valid k colors given as input.

II. STATE OF THE ART

Grover’s algorithm is a quantum search algorithm that provides a quadratic speedup over classical search methods. It operates by iteratively applying an oracle to mark invalid states and an amplification operator to increase the probability of observing valid solutions. In the context of k-coloring, the oracle is responsible for validating color assignments and marking those that violate the adjacency constraint.

Despite its theoretical advantages, Grover’s algorithm faces practical limitations:

- **Oracle Complexity:** Traditional implementations require multiple control blocks for the invalid color assignment, leading to higher circuit depth.
- **Scalability:** As the graph size or the number of colors (k) increases, the resource requirements grow significantly, making the algorithm unnecessarily complex for large instances.
- **Hardware Constraints:** Current quantum devices are limited in the number of qubits and allowable gate depth, which restricts the implementation of complex oracles.

Recent research has explored various strategies to optimize Grover’s algorithm for graph coloring. These include amplitude amplification, which reduces the number of iterations required, and problem-specific oracle designs that minimize the number of gates. However, these approaches often trade

off generality for efficiency, making them less applicable to diverse problem instances.

III. METHODOLOGY

A. Problem Definition

The k -coloring problem involves a graph $G = (V, E)$ with $|V|$ vertices and $|E|$ edges. The task is to assign a color $c(v)$ to each vertex $v \in V$ such that:

$$c(u) \neq c(v), \quad \forall (u, v) \in E \quad (1)$$

Since the representation of each color in the problem is binary, given as input k (number of different colors), this is represented in binary with $\lceil \log_2 k \rceil$ bits, immediately creating the problem of invalid colors in the graph: taking $\lceil \log_2 k \rceil$ means that we're leaving some bits unused in the binary representation, for example: if $k = 3$ (3 possible colors to use), we have that $\lceil \log_2 k \rceil = 2$, so we represent these 3 colors with 2 bits, first color = 00, second color = 01, third color = 10.

Now, what about 11? We need to prevent the circuit from using this invalid configuration of bits that does not represent any valid color. This is called, indeed, an **invalid color**.

In the original approach described in [1], invalid colors are marked by repetitively applying masks to the input configurations. These masks are designed to flip an ancilla qubit if any of the masked configurations result in a state of all ones. For example, if $k = 5$ (binary representation: 100, that is the fifth color starting from 000), one invalid color is 6 (binary representation: 101). To mark this invalid color, a mask is applied to transform the state 110 into 111. This transformation enables the use of a multi-controlled X (MCX) gate to flip the ancilla qubit when all qubits are 1.

The mask essentially performs bitwise operations on the input qubits, using X gates to invert bits as necessary and Z gates to preserve certain configurations. Below is the corresponding quantum circuit that represents the described mask:

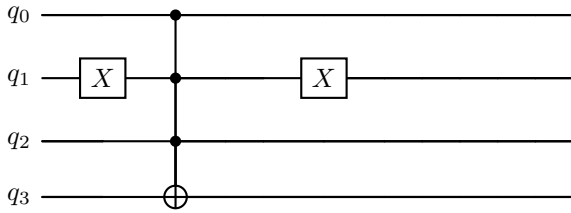


Fig. 1. Example of a MCX gate applied to an ancilla in order to mark the invalid configuration 101 (6th color)

This approach works because the mask transforms the binary representation 110 into 111. When the transformed configuration becomes 111, the multi-controlled X (MCX) gate flips the ancilla qubit, marking the invalid state.

Remember, in quantum computing, all computational basis states coexist in superposition, and quantum operations act on all of them simultaneously. However, each basis state evolves

independently under the same sequence of gates, according to how those gates act on its bit pattern. When the ancilla qubit is flipped for one particular basis state, it remains unchanged for all the others.

By applying this concept to any invalid color in the graph coloring problem, we can mark one ancilla qubit for each node.

Why one for each node? Because, as explained earlier, when the ancilla qubit is flipped for a particular basis state, it remains unchanged for all others. This allows different configurations to flip their corresponding ancilla qubits independently within the same superposition. As a result, each node's ancilla qubit is flipped at most once, which simplifies the marking process and ensures consistency across all nodes.

The first thing that we notice, though, is that we need multiple checks, one for each invalid color in the circuit, ending up with $2^{\lceil \log_2(k) \rceil - 1} - k$ controls.

B. Range Oracle Design

The Range Oracle is designed to address the inefficiencies of traditional oracle implementations. Instead of validating each invalid color individually, the Range Oracle performs a single range comparison to mark colors that exceed a predefined threshold. This reduces the number of oracle operations and simplifies the circuit design.

The algorithm for the Range Oracle is as follows:

Function `invalid_color_greater_than`
`(qc, problem, color, i, dest){`

```

    a ← node_qubits(problem, i)
    color ← color - 1
    num_binary ← bin(color)[2:]
    if num_binary[0] = "0" then
        qc.cx(a[-1], dest)
        qc.x(a[-1])
    end if
    for position, value in num_binary[1:] do
        relevant_qubits ← a[-(position+1):]
        if value = "0" then
            qc.mcx(relevant_qubits, dest)
            qc.x(a[-(position+1)])
        end if
    end for
}
```

The concept behind this implementation involves modifying the traditional less-than oracle approach described in [2]. In that work, a fully functional less-than oracle is presented, which flips the phase of all computational basis states representing numbers less than a specified reference value k . For instance, if $k=5$, the oracle marks by phase-flipping the states corresponding to the binary representations of 1, 2, 3, and 4. These marked states are subsequently amplified through the amplitude amplification phase of Grover's algorithm, increasing their probability.

In the referenced work [2], the implementation of the greater-than oracle was achieved indirectly by applying a

less-than oracle and performing an additional phase flip to mark states satisfying the greater-than condition. While this approach suffices for phase-flipping operations, my implementation required a standalone greater-than oracle. This was necessary because my design needed to adapt the greater-than condition to flip an ancilla qubit, rather than flipping the phase of the state.

This modification was crucial for my specific application, where the ancilla qubit serves as a marker for later operations, enabling more flexibility in the construction of the quantum circuit.

Following this reasoning, I developed the "special" greater-than oracle shown above.

Below it's possible to see how it works with an example using $k = 5$ colors in just one node (in the real case the oracle is applied in the same way to all the other nodes of the graph, and for any number of k and qubits used to represent it):

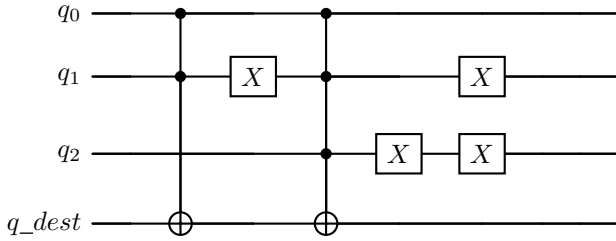


Fig. 2. Example of a greater than oracle marking the ancilla when the state becomes greater than 100

The last layer of X gates is added following the same structure presented in [2], in order to flip back the qubits to their original state.

C. Implementation Details

The implementation consists of the following components:

- **Oracle Module:** Implements the Range Oracle and integrates it with the quantum circuit.
- **Graph Representation:** Encodes the graph structure and adjacency constraints.
- **Circuit Construction:** Builds the quantum circuit, including initialization, Grover iterations, and measurement.
- **Simulation and Validation:** Runs the circuit on classical simulators to validate correctness and measure performance.

IV. RESULTS AND DISCUSSION

A. Performance Metrics

The Range Oracle was evaluated using graphs of varying sizes and color counts. Key metrics include:

- **Circuit Depth and Width:** For a graph with 3 nodes and 5 colors, the circuit achieved a depth of 72 and a width of 16, resulting in a depth x width product of 1152, compared to the $92 \times 22 = 2024$ of the original algorithm.

- **Oracle Efficiency:** The Range Oracle reduced oracle operations to, in the worst case, $O(\log_2(k))$ per node, compared to $O(2^{\lceil \log_2(k) \rceil} - k)$ in traditional methods.
- **Scalability:** Resource requirements exhibited sub-linear growth with increasing k , demonstrating improved scalability.

B. Limitations

While the Range Oracle improved resource efficiency, certain limitations remain:

- **Hardware Constraints:** Current quantum devices impose practical limits on qubits and coherence times.
- **Scalability:** Further validation is needed for larger graphs to confirm the approach's effectiveness.
- **Performance Issues:** Due to probable algorithm inefficiencies or secondary errors, the probability of the correct solutions is less amplified than the traditional approach, leading to less accuracy.

V. CONCLUSION AND FUTURE WORK

This paper presented a Range Oracle methodology for solving the k -coloring problem using Grover's algorithm. While the approach is not transformative, it offers a practical and modest improvement in oracle complexity, circuit depth, and scalability.

Future research directions include:

- Testing larger graphs and higher k values to evaluate scalability.
- Exploring hybrid quantum-classical approaches to optimize resource usage.
- Implementing the methodology on real quantum hardware to assess its practical feasibility.
- Revision of the possible errors in the algorithm in order to produce a less noisy circuit and make it more accurate, resulting in a higher probability of correct solutions.

The final performance of the circuit, given the background noise, is still promising. Here we can clearly see the plot describing how each solution is amplified for $k = 5$:

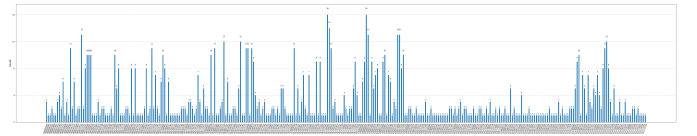


Fig. 3. probabilities for each configuration for $k = 5$

VI. CODE AND DATA

Quantum k -Coloring Repository, Arcangeli Lorenzo.

REFERENCES

- [1] O. Belletti, "Analyzing, Fixing, and Optimizing a Space-Efficient Quantum Circuit for the Graph K-Coloring Problem," GitHub repository, 2025. [Online]. Available: K-coloring Repository, Oscar Belletti.
- [2] Javier Sanchez-Rivero, Daniel Talaván, Jose Garcia-Alonso, Antonio Ruiz-Cortés, Juan Manuel Murillo, "Less-than operator quantum circuit for Grover's algorithm: GitHub repository. Available: JSRivero less-than oracle Repository.